

닥터그린 스마트팜 가상 실습실 — 수업 가이드라인

교사용 운영 매뉴얼 · 120분 · 고등학생 대상 · 제작: 서울로봇인공지능과학관

작성일: 2026-07-09

1. 수업 개요

항목	내용
수업명	닥터그린 스마트팜 가상 실습실 — "나만의 스마트팜 앱 완성하기"
대상	고등학생 15명 (1인 1PC)
시간	120분 (중간 휴식 10분 포함)
장소	컴퓨터실 (학생 PC 15대 + 교사 PC 1대 + 프로젝터)
실습 URL	https://doctor-green-edu.vercel.app (접속 시 자동으로 실습실로 이동)
준비 프로그램	크롬(Chrome) 최신 버전, 인터넷 연결 필수 (STEP 1은 실시간 API 호출)
실습 형태	STEP 1은 학생이 인증키를 붙여넣어 잠긴 날씨 기능을 여는 실습으로, 내 위치 기반 실제 기상청 날씨 가 표시된다. STEP 2~4는 가상 시뮬레이션. 코드 타이핑 없음 — 유일한 입력은 인증키 붙여넣기 한 번
제작	서울로봇인공지능과학관

수업 한 줄 소개: 스마트팜을 이루는 4가지 기술(공공데이터 API(실제 호출), AI 비전, IoT 센서, 카메라 영상)을 하나씩 직접 연결하고, 그 데이터가 데이터베이스(가상 Supabase)에 쌓여 **하나의 스마트팜 앱이 완성되는 과정**을 체험한다.

수업 구조: 학생은 방 번호(1~15번)를 배정받아 자기 방에서 4개의 STEP(스테이션)을 순서대로 완료한다. 각 STEP을 완료할 때마다 "내 앱"의 잠긴 기능이 하나씩 켜지고, 4개를 모두 연결하면 앱이 완성된다.

2. 학습 목표

수업이 끝나면 학생은 다음을 설명·체험할 수 있다.

1. **공공데이터 API:** 공공데이터포털(data.go.kr)에서 인증키를 발급받아 인증하는 과정을 체험하고, 기상청 초단기실황으로 내 위치의 진짜 날씨 데이터를 받아본다.
2. **AI 비전(YOLO):** 라벨링된 이미지로 신경망을 반복 학습(epoch)시키면 loss는 내려가고 정확도(accuracy)는 올라간다는 것.
3. **IoT 센서:** ESP32 같은 소형 장치가 온도·습도 값을 실시간으로 데이터베이스에 전송하고, 앱이 이를 그래프로 보여주는 흐름.
4. **카메라 스트리밍:** WiFi 카메라가 영상을 클라우드에 올리고 앱이 주소로 실시간 표시하는 원리.
5. **데이터베이스:** 모든 데이터가 테이블에 INSERT(행 추가)되어 쌓이고, 앱의 기능들은 결국 "테이블에서 데이터를 읽어 보여주는 것"이라는 큰 그림
6. **보안 기초:** 인증키·카메라 주소(camera_url) 같은 '열쇠'는 비밀번호처럼 비밀로 관리해야 한다는 것.

2-1. 2022 개정 교육과정 연계 (참고)

과목·영역	이 수업의 연계 요소
정보 — 인공지능의 이해(기계학습·데이터)	STEP 2 라벨링·epoch·loss/accuracy, 오탐·과적합
인공지능 기초 — 기계학습과 데이터, 인공지능의 사회적 영향	STEP 2, 심화 모듈(판단보류·가드레일·AI 윤리)
정보 — 디지털 문화(정보보호·보안)	인증키·camera_url 보안
수학(확률과 통계) — 조건부확률	미니 혼동행렬·정밀도 활동
기술·가정/농업생명과학 — 스마트 농업 기술	전체 시스템, IoT 센서, 자동화 규칙
진로와 직업	진로 연계 코너(\$12)

위 연계는 과목·영역 수준의 참고 매핑이며, 학교별 교육과정 편성에 따라 조정한다.

3. 사전 준비 체크리스트 (교사)

수업 전달

- ☐ 배포 서버에 기상청 인증키 설정 (필수·전날까지!) — 실제 날씨 호출은 서버에 저장된 키로 이뤄진다. ① `vercel env add KMA_SERVICE_KEY`로 키 등록 → ② 재배포 (환경변수는 재배포해야 반영됨!) → ③ 실습 URL의 STEP 1에서 실제 날씨가 뜨는지 직접 리허설. 카드 ③에 오류 문구가 뜨면 서버 키 문제다. 미설정 시 전 학생이 '체험 모드(가상 날씨)'로만 진행된다.
- ☐ 학생 PC 크롬의 위치 권한 허용 여부 확인 (거부되어도 5개 지역 버튼으로 실제 호출 가능하므로 수업 진행에는 지장 없음)
- ☐ 학생 PC 15대 전부에서 실습 URL 접속 테스트 (크롬 권장)
- ☐ 프로젝터 + 교사 PC에서 시연용 접속 확인
- ☐ 좌석별 방 번호 배정표 작성 (1번 자리 → 방 1, ... 15번 자리 → 방 15)
- ☐ 실물 시연용 장비 준비: 헥사보드(ESP32) 딸기 디바이스, 실제 닥터그린 앱

수업 직전 — 초기화 관련 중요 사항

- ☐ 이전 수업 기록이 남아 있으면 초기화 필요. 데이터는 각 PC의 브라우저(localStorage)에 저장되므로, 교사 PC에서 초기화 버튼을 눌러도 학생 PC는 초기화되지 않는다.
- ☐ 수업 도입 때 학생 각자 자기 PC에서 "이 방 초기화 (처음부터 다시)" 버튼을 누르게 안내

알아두면 좋은 운영 특성

- **교사 화면에서 학생 진행 상황이 원격으로 보이지 않는다** (진행도는 각 PC 브라우저에만 저장됨). 순회 지도가 필수이며, 각 학생 허브 화면 상단의 진행도 바(0/4 → 4/4)로 확인한다.
- 다른 PC라면 같은 방 번호를 골라도 데이터가 충돌하지 않지만, 혼동 방지를 위해 배정표대로 진행한다.
- **STEP 1(날씨)은 실시간 인터넷 연결이 필수다.** STEP 2~4는 첫 로드 후 브라우저 안에서 동작하므로, 네트워크가 불안정한 교실이라면 수업 시작 시 각 스테이션 페이지를 한 번씩 미리 열어두면 안전하다.
- **위치 권한 관련 개인정보:** 학생 위치(위경도)는 기상청 격자 좌표(nx/ny) 변환과 지역 이름 표시에만 그 순간 사용되며 저장되지 않는다 — weather_cache 테이블에 남는 것은 지역명·기온·하늘 상태뿐이고, 서버도 좌표를 기록하지 않는다. 학생·학부모 문의 시 이렇게 안내하면 된다.

4. 수업 타임라인 (120분)

시간	구간	내용
00:00-10:00 (10분)	도입	스마트팜 소개, 수업 목표 안내, URL 접속 + 방 번호 선택 + 방 초기화
10:00-25:00 (15분)	STEP 1	공공데이터 날씨 연결 (배포 서버에 기상청 키 설정 필요)
25:00-55:00 (30분)	STEP 2	AI 비전(YOLO) 학습 — 수업의 핵심, 가장 길게
55:00-65:00 (10분)	휴식	—
65:00-85:00 (20분)	STEP 3	IoT 센서 연결
85:00-95:00 (10분)	STEP 4	카메라(영상) 연결
95:00-110:00 (15분)	앱 완성	내 앱 확인 + 캡스톤 '자동화 규칙 만들기' (+실물 디바이스 시연)
110:00-120:00 (10분)	마무리	마무리 퀴즈(인앱 7문항), 소감 발표, 진로 연계

5. 도입 (10분) — 진행 시나리오

1. **동기 유발 (3분):** "농장에 사람이 없어도 온도를 재고, 병든 잎을 찾아내고, 물을 주는 농장이 있다면?" — 스마트팜 사진/영상 1~2장. 실물 딸기 디바이스가 있다면 이때 잠깐 보여주고 "이것과 똑같은 구조를 오늘 여러분이 직접 만든다"고 예고.
2. **수업 구조 안내 (2분):** 오늘 4개의 기술 스테이션을 통과할 때마다 내 앱의 기능이 하나씩 켜진다. 마지막에 4/4가 되면 나만의 스마트팜 앱 완성.
3. **접속 (5분):**
 - 크롬에서 `doctor-green-edu.vercel.app` 접속
 - "스마트팜 가상 실습실" 화면에서 **자기 방 번호 선택** (배정표대로)
 - "**이 방 초기화 (처음부터 다시)**" 한 번 눌러 깨끗한 상태로 시작

- 교사가 견본 인증키를 공유 — STEP 1에서 붙여넣기용 (학생 각자 발급은 data.go.kr 회원가입·활용신청 시간이 필요하므로 수업 중에는 불가, 원하면 사전 과제로)
- 화면 구성 설명: 상단 진행도 바(0/4), 4개의 STEP 카드, 하단의 가상 Supabase 테이블 4개

도입 멘트 예시: "오늘 타이핑은 인증키 붙여넣기 한 번뿐입니다. 하지만 버튼을 누를 때마다 오른쪽 화면에 진짜 개발자들이 쓰는 코드와 데이터베이스가 실시간으로 움직이는 걸 보게 됩니다. 그게 오늘의 관전 포인트예요."

6. STEP 1 — 공공데이터 날씨 연결 (15분)

목표: 인증키를 붙여넣어 잠겨 있던 날씨 기능을 열고, 기상청 초단기실황으로 내 위치의 진짜 날씨를 받아와 그 데이터가 DB 테이블(weather_cache)에 쌓이는 것을 관찰한다.

학생 조작 순서

1. 허브에서 "공공데이터 날씨 연결" 카드 클릭. 이때 오른쪽 카드 ③은 "잠김" 상태 — "아직 인증키가 없어서 날씨 기능이 작동하지 않아요"라는 문구와 함께 "내 위치 날씨 가져오기" 버튼이 회색으로 눌러지지 않는다
2. 카드 ① 입력란("일반 인증키(Decoding)를 여기에 붙여넣기")에 인증키를 붙여넣고 "인증키 저장" 클릭 → 버튼이 "인증키 확인 중..."으로 바뀌고 약 1초 뒤 "저장된 인증키 · N번 방"이 뜬다. 바로 이 순간 카드 ③이 "잠김"에서 "켜짐"으로 살아난다(방별 저장이라 새로고침 후 재입력 불필요, "키 변경" 버튼으로 교체 가능)
3. 카드 ② "이 키가 실제 코드에 들어가요": 코드 창에 호출 URL(`getUltraSrtNcst?serviceKey=...` — 키는 마스킹 표시)이 자동으로 채워지는 것 확인
4. 카드 ③ "내 위치 날씨 가져오기" 클릭 → 브라우저 위치 권한 허용 → "내 위치 확인 중..." → "기상청에서 받아오는 중..." → 실제 기온·하늘 상태 카드가 표시되고 `weather_cache` 테이블에 INSERT, 완료 배너 "홈 화면의 날씨 기능이 켜졌어요!"
5. 위치 권한이 어려우면: "위치 권한이 어려우면 지역을 골라 호출하기" → 서울/대전/광주/부산/제주 버튼 (이 경로도 실제 기상청 호출)
6. 허브 진행도 1/4

관찰 포인트 (교사가 짚어줄 것)

- 잠김→켜짐 전환이 이 스테이션의 하이라이트: 키를 넣기 전엔 버튼이 죽어 있다가 저장하는 순간 기능이 살아난다 — "인증이 곧 문을 여는 열쇠"라는 개념의 시각적 클라이맥스. 저장 전후를 일부러 비교해 보여줄 것.
- 화면에 뜬 기온을 지금 창밖·핸드폰 날씨 앱과 비교해 보게 할 것 — "진짜 데이터"임을 확인하는 이 수업 최고의 리얼리티 순간.
- 호출 URL의 nx/ny는 기상청 격자 좌표 — 위경도를 기상청 예보 지도의 칸 번호로 변환한 것.
- 진짜 데이터가 도착해도 저장 연출은 가상 Supabase — "데이터가 테이블에 쌓이는 구조"는 실제와 동일하다.

교사만 알아둘 것 (연출 원리): 실제 기상청 호출은 서버에 미리 저장된 키가 수행하고, 학생이 넣는 키는 잠긴 기능을 여는 '스위치' 역할이다. 그래서 학생이 아무 키나(발급 지연·오타 포함) 형식만 맞게 넣어도 수업이 막히지 않고 실제 날씨가 뜬다. 학생에겐 굳이 설명하지 않아도 되며, "제가 아무 글자나 넣어도

되던데요?"라고 물으면 "형식이 맞으면 앱이 받아들이도록 만들어 수업이 안 끊기게 한 것"이라고 답하면 된다.

예상 질문과 답

- Q. "진짜 오늘 날씨예요?" → A. 네. 기상청 초단기실황, 방금 관측된 실제 값입니다(매시간 갱신).
- Q. "왜 친구랑 온도가 조금 달라요?" → A. 위치(격자 칸)가 다르면 관측값이 다릅니다.
- Q. "인증키는 왜 필요해요?" → A. 누가 얼마나 쓰는지 관리하고, 서버 과부하를 막기 위해. 실제 공공데이터포털(data.go.kr)에서 누구나 무료 발급 가능.
- Q. "인증키를 친구에게 알려줘도 돼요?" → A. 안 됩니다. 인증키는 비밀번호와 같아서 남이 내 이름으로 서버를 호출할 수 있습니다. 실제 개발 현장 보안 사고 단골이 '코드나 GitHub에 키를 그대로 올리는 것' — 그래서 키는 코드가 아니라 서버 환경변수 같은 숨긴 곳에 보관합니다. (오늘 견본 키를 다 같이 쓰는 건 수업용 특례라는 점도 짚어줄 것)

심화 활동 (빨리 끝낸 학생)

- 5개 지역 버튼을 전부 눌러 **실제 지역별 기온 비교표** 만들기. (부록 B 활동지 사용)
- 옆 친구와 "내 위치" 값 비교해 보기.

문제 시: 카드 ③에 오류 문구("지금 서버에 기상청 인증키가 준비되어 있지 않아요..." 또는 "지금 기상청에서 실제 날씨를 받아오지 못했어요...")와 함께 "**체험 모드로 진행 (가상 날씨)**" 버튼이 뜨면, 이는 **학생 키 문제가 아니라 서버 키 미설정·기상청 장애·네트워크 오류** 신호다. 교사가 서버 키(\$3)·네트워크를 점검하고, 급하면 체험 모드로 수업을 계속한다.

7. STEP 2 — AI 비전(YOLO) 학습 (30분, 수업의 핵심)

목표: 라벨링 → 학습(epoch 반복) → 추론이라는 AI 학습의 3단계 전체를 체험하고, loss/accuracy 그래프를 해석한다.

도입 멘트: "AI는 태어날 때부터 병든 잎을 아는 게 아니에요. 사람이 '이건 건강, 이건 병해'라고 정답을 알려준 사진(라벨링)을 수십 번 반복해서 보여주면(epoch) 점점 실수가 줄어듭니다(loss). 오늘 여러분이 직접 AI 선생님이 됩니다."

학생 조작 순서

1. **" 샘플 잎 사진 100장 불러오기"** 클릭 → 잎 사진 100장이 10×10 그리드로 표시(스크롤). 라벨링되면 사진마다 색 테두리가 표시되고, 하단에 범례(건강한 잎/병해 의심/해충 흔적)가 나타난다
2. **"반복 횟수 (epoch)"** 슬라이더 조절 (5~30, 기본 15) — 처음엔 기본값 15 권장
3. **"학습 시작"** 클릭 → 3단계가 자동 진행됨:
 - **라벨링** (약 11초, 장당 0.1초): 사진마다 건강/병해/해충 색깔 테두리가 붙는다
 - **학습**: 오른쪽 그래프에서 **loss(빨강)**는 내려가고 **accuracy(초록)**는 올라가는 곡선이 실시간으로 그려진다. 왼쪽 가중치 격자(50칸)의 색이 변하는 것도 관찰
 - **추론 준비 완료**

4. "새 잎 사진 진단하기" 클릭 → 작물(토마토/고추/상추/딸기/오이)과 진단 결과(건강/병해/해충) + 신뢰도(%)가 나오고 **diagnoses** 테이블에 INSERT
5. 완료 배너 "AI 진단 기능이 켜졌어요! (정확도 XX%)" 확인 → 진행도 2/4

관찰 포인트

- **loss/accuracy 그래프가 이 수업 최고의 교육 장면.** 전체 학생을 잠깐 멈추게 하고 프로젝터로 함께 해석할 것: "실수(loss)는 줄고, 정답률(accuracy)은 올라간다. 이게 '학습'의 정체입니다."
- 신뢰도가 100%가 아닌 80~97%인 이유: AI는 항상 확신이 아니라 '확률'로 답한다.
- epoch를 늘려도 정확도는 어느 순간부터 거의 오르지 않는다(수확체감). 앱 그래프는 하락까지는 보여주지 않으므로, 실제 AI에서는 지나친 반복이 '과적합(암기)'으로 이어져 처음 보는 사진에 오히려 약해질 수 있다는 것을 교사가 말로 짚어줄 것 (마무리 퀴즈에 출제됨).
- 진단을 여러 번 반복하면 건강한 잎을 '병해'로 잘못 판정하는 경우가 나온다 — 이것이 '오탐(거짓양성, FP)'. 마무리 퀴즈에 나오는 용어이므로 한 번 소리 내어 정리할 것.
- **실제 앱은 신뢰도 75% 미만이면 판단을 보류한다** — '좋은 AI는 모를 때 모른다고 말한다'. 심화 활동까지 못 가는 학생도 있으므로 전체 학생에게 한 번 소리 내어 짚어줄 것 (마무리 퀴즈 6번에 출제됨).

예상 질문과 답

- Q. "epoch을 30으로 하면 뭐가 달라져요?" → A. 더 오래 공부하는 것. 대체로 정확도가 오르지만, 실제 AI에서는 너무 반복하면 '과잉 학습(암기)' 문제도 생긴다.
- Q. "100장으로 충분해요?" → A. 수업용 시뮬레이션 기준이라 100장. 실제 YOLO 모델은 수천~수만 장을 쓴다.
- Q. "라벨링을 사람이 다 해요?" → A. 그렇다. 그래서 '데이터 라벨러'라는 직업도 있다.

심화 활동

- **epoch 실험:** 이 방 초기화 후 epoch 10으로 다시 학습 → 20/30일 때와 최종 정확도 비교. "반복 횟수와 정확도의 관계"를 스스로 발견하게 한다.
- 진단을 여러 번 눌러 다양한 작물/결과가 **diagnoses** 테이블에 쌓이는 것 관찰.
- **"AI를 속여라" 챌린지:** 새 잎 진단을 여러 번 반복하며 신뢰도가 낮게(80% 미만) 나오는 케이스를 수집하고, 왜 낮은지 가설을 세운다. "실제 닥터그린 앱은 신뢰도 75% 미만이면 판단을 보류한다"는 실제 동작과 연결하면 '신뢰도 = 확률(확신의 정도)'이라는 관찰 포인트를 몸으로 이해하게 된다.
- **미니 혼동행렬 만들기:** 진단을 10회 눌러 결과를 '맞음/틀림' 표로 집계한다. (예: 건강한 잎을 병해로 잘못 본 횟수 = 오탐) 정밀도(precision) 개념을 맛보는 활동. 표 양식 한 줄 예시: | 회차 | 실제 (내가 본 잎) | AI 판단 | 신뢰도 (%) | 맞음/틀림 | 정밀도 = "병해라고 판정한 것 중 실제 병해의 비율" — 수학의 조건부확률과 이어지는 개념임을 언급하면 좋다. (부록 B 활동지 사용)

8. STEP 3 — IoT 센서 연결 (20분)

목표: IoT 장치(가상 ESP32)의 센서 값이 데이터베이스로 전송되어 실시간 그래프가 되는 흐름을 체험한다.

도입 멘트: "화분 옆에 붙은 작은 컴퓨터(ESP32)가 1.5초마다 온도·습도를 재서 WiFi로 데이터베이스에 보냅니다. 오늘은 여러분 손가락이 온도계 역할을 합니다."

학생 조작 순서

1. "전원 켜기" 클릭 → 센서 보드 상태가 ●에서 ●으로
2. 온도(0~40°C)와 습도(0~100%) 슬라이더를 움직여 값 설정
3. "측정값 1회 전송" 클릭 → 화면의 흐름도에서 데이터 점이 센서→Supabase로 이동하는 애니메이션 → `sensor_readings` 테이블에 INSERT (이 순간 스테이션 연결 완료, 진행도 3/4)
4. "▶ 자동측정 시작" 클릭 → 1.5초마다 자동 전송, 오른쪽 실시간 그래프가 살아 움직임
5. 자동측정 중 "입김 불기 (습도 ↑)" 클릭 → 습도가 확 뛰면서 그래프에 산봉우리가 생김

관찰 포인트

- 흐름도 애니메이션: "센서 → 인터넷 → 데이터베이스 → 앱 그래프"가 한 화면에 보인다. 식물 스마트팜과 완전히 같은 구조.
- 전송값에 작은 노이즈(오차)가 섞이는 것: 실제 센서도 항상 미세하게 흔들린다.

예상 질문과 답

- Q. "실제로도 슬라이더로 조절해요?" → A. 아니다. 실제로는 DHT11 같은 센서 부품이 진짜 공기를 잴다. 슬라이더는 '자연' 역할의 시뮬레이션.
- Q. "1.5초마다 보내면 데이터가 너무 많아지지 않아요?" → A. 좋은 질문. 그래서 실제 시스템은 전송 주기를 조절하고 오래된 데이터를 정리한다.

심화 활동

- 자동측정을 켜 채 온도 슬라이더를 천천히 올렸다 내리며 그래프에 파도 모양 만들기.
- 테이블의 `device_id` 값 확인 — 장치마다 고유 ID가 있어 여러 화분을 구분한다는 것 발견.
- "이상치 탐정" 게임: 짝과 자리를 바꿔, 상대가 자동측정 중 몰래 넣은 '입김 이벤트(습도 급상승)'가 몇 시 몇 분에 있었는지 실시간 그래프만 보고 찾아낸다. 기존 '파도 만들기'의 역방향 활동으로 준비물 없이 바로 가능하며, "데이터에서 사건을 읽어내는" 분석 경험을 준다.

9. STEP 4 — 카메라(영상) 연결 (10분)

목표: WiFi 카메라가 클라우드에 영상을 올리고, 앱이 주소(camera_url)로 실시간 표시하는 원리를 체험한다.

도입 멘트: "멀리 있는 농장을 핸드폰으로 보려면? 카메라가 영상을 클라우드에 올리고, 앱은 그 '주소'만 알면 됩니다."

학생 조작 순서

1. "카메라 연결하기" 클릭 → 카메라 주소(camera_url)가 발급되고 코드 창에 자동으로 채워짐
2. "라이브 켜기" 클릭 → 오른쪽 프리뷰에 LIVE 배지와 함께 가상 라이브 화면 시작 (진행도 4/4 달성!)

3. "스냅샷 한 장 더 찍기" 클릭 → `camera_frames` 테이블에 프레임 파일(frame_0001.jpg...)과 감지 내용("딸기 잎 감지", "딸기 꽃 감지", "열매 감지", "포기 전체" 중 하나)이 INSERT
4. "스냅샷 한 장 더 찍기"를 3~4회 반복 → `camera_frames` 테이블의 `note` 열(네 가지: "딸기 잎 감지", "딸기 꽃 감지", "열매 감지", "포기 전체")을 시간순으로 읽어 "오늘 농장에 무슨 일이 있었나"를 한 문장으로 재구성해 발표 (감지 로그 타임라인 활동)

관찰 포인트

- 테이블의 `note` 열: "딸기 잎 감지", "딸기 꽃 감지", "열매 감지", "포기 전체" 네 가지 — STEP 2에서 배운 AI 비전이 카메라 영상과 결합되면 '자동 감시'가 된다는 연결고리를 짚어줄 것.
- `camera_url`도 '열쇠': 주소가 새면 남이 내 농장을 실시간으로 본다. 실제로 초기 비밀번호를 바꾸지 않은 가정용 IP카메라가 해킹되어 영상이 유출된 사건들이 있었다 — STEP 1 인증키와 같은 원리로 주소·키는 비밀로 관리해야 함을 짚어줄 것.

예상 질문과 답

- Q. "영상 전체가 DB에 저장돼요?" → A. 보통 영상 자체는 스토리지에, DB에는 프레임 파일 이름과 감지 결과 같은 '기록'만 저장한다.

심화 활동

- **연결고리 짚기:** STEP 2에서 배운 AI 비전이 이 카메라 영상과 결합되면 '사람 없이 자동 감시'가 된다는 것을 확인한다. (뒤에 이어질 자동화 규칙 설계의 연습이 됨)

10. 앱 완성 + 캡스톤 (15분)

내 앱 확인 (전원 공통, 4분)

1. 허브 우상단 "내 앱" 버튼 클릭
2. 폰 크기 목업에서 4개 기능이 모두 켜진 것 확인: 오늘 날씨 / AI 진단(신뢰도·작물) / 실시간 분석(온습도+그래프) / 실시간 카메라(LIVE)
3. **아직 STEP을 못 끝낸 학생은 잠금 카드가 보인다** — 잠금 카드의 링크를 눌러 해당 STEP으로 이동해 마저 완료하게 한다 (이 시간이 자연스러운 보충 시간)
4. 정리 멘트: "여러분이 오늘 연결한 4개의 기술이 모여 이 앱 하나가 됐습니다. 세상의 모든 앱이 이런 식으로 여러 기술과 데이터의 조합입니다."

캡스톤 — 스마트팜 자동화 규칙 만들기 (8분, 종합 활동)

- **위치:** "내 앱" 화면 하단의 "스마트팜 자동화 규칙 만들기" 섹션.
- **활동:** 조건("만약 ~이면") + 행동("~한다")을 골라 규칙 추가. 조건 10종은 오늘 연결한 4개 데이터(날씨·AI·센서·카메라)에서 나오고, 규칙마다 데이터 출처 배지가 붙는다. 행동은 물 주기·환기팬·알림·차광 커튼 등 6종.
- **목표:** 규칙 3개 이상 + 서로 다른 데이터 3종 이상 사용 (화면의 체크리스트로 확인).
- **규칙 시뮬레이터:** 가상 상황(하늘·기온·습도·AI 판정)을 바꿔가며 내 규칙이 실제로 발동하는지 확인.

- **짚어줄 것:** "여러분이 방금 만든 것이 프로그래밍의 조건문(if → then)입니다. 오늘 연결한 4개 데이터가 '조건'이 되는 순간, 사람이 없어도 스스로 판단하는 스마트팜이 됩니다 — 수업 처음에 던진 질문의 답이에요."
- **물 주기 규칙:** 공기 습도가 아니라 토양수분('토양수분이 30% 아래로 내려가면(마름)')으로 만들게 안내할 것 — 실제 농업에서 관수 판단 기준은 흙의 수분이며, 심화 모듈(\$11)의 결합 진단과 이어진다.
- **발표(1~2명):** 자기 규칙 하나를 고르고 '왜 그 조건에 그 행동인지' 근거를 설명하게 한다. (근거를 말하게 하는 것이 핵심 — 고등학생 수준의 설계 판단 연습)
- **예상 질문:** Q. "조건 두 개를 동시에 걸 수 없나요?" → A. 좋은 질문. 실제 시스템은 AND/OR로 조건을 조합한다. 심화 모듈의 '결합 판정'이 바로 그 예.

실물 디바이스 시연 (3분, 있을 때만)

실물 hexa보드(ESP32) 딸기 디바이스가 있다면:

- 보드의 온습도 센서(DHT11)·토양수분 센서·LED를 보여주고, LCD에 표시되는 실시간 온습도 확인
- 실제 닥터그린 앱의 실시간 화면을 띄워 **진짜 Supabase에 2초마다 데이터가 쌓이는 것** 시연

11. 심화 모듈 — “두 데이터를 합치면”: 결합 진단과 AI 종합 소견

심화·후속 차시용 (약 25~30분). 기본 4-STEP을 마친 학생 대상. 이 모듈은 가상 실습실이 아니라 실제 닥터그린 앱의 카메라 진단 화면으로 진행한다(교사 시연 또는 학생 체험). STEP 2에서 배운 AI 비전(잎 사진으로 병을 봄)과 STEP 3에서 배운 IoT 센서(흙의 물기를 잴)를 하나로 합치면, 어느 하나만으로는 알 수 없던 ‘과습·물부족’까지 판정할 수 있음을 체험한다.

한 줄 소개: “잎만 보면 노랑다는 것만, 흙만 보면 젖었다는 것만 안다. 둘을 합쳐야 비로소 ‘과습’이라고 말할 수 있다.” — 이것이 이 모듈의 핵심이다.

학습 목표

- 결합(멀티모달): 서로 다른 데이터(사진 + 센서 수치)를 합치면 각각만으로는 불가능한 판단이 가능해진다.
- 규칙 기반 판정: ‘비전 증상 × 토양수분’의 조합표로 원인(과습 / 물부족 / 병해 / 영양의심)을 가려낸다.
- 판단보류(불확실성): 신호가 서로 엇갈리면 시스템은 억지로 답하지 않고 ‘판단보류’한다.
- 규칙 AI vs 생성형 AI: 정해진 표로 계산하는 판정(💧)과, 그 위에서 사람 말로 소견을 쓰는 생성형 AI(🌿)의 차이.
- 믿을 수 있는 AI: ‘없는 값을 지어내지 않기’, ‘센서가 응답 없으면 그렇다고 말하기’ 같은 규칙(가드레일)이 왜 필요한가.

도입 멘트

“STEP 2에서 AI는 잎이 노랑다는 걸 알아냈죠. STEP 3에서 센서는 흙이 젖었다는 걸 췄고요. 그런데 ‘노란 잎’만으로 물을 너무 많이 준 건지, 영양이 부족한 건지 알 수 있을까요? 알 수 없습니다. 하지만 여기에 ‘흙이 젖었다’를 합치면? ‘아, 물을 너무 많이 줘서(과습) 뿌리가 숨을 못 쉬어 잎이 노랗진 거구나’를 알 수 있어요. 오늘은 두 데이터를 합치는 순간 무슨 일이 벌어지는지 봅시다.”

진행 시나리오 (교사 시연 또는 학생 체험, 약 25~30분)

- ① 문제 제기 (5분): “잎만 보면? 흙만 보면?” 각각의 한계를 던진다. 노란 잎 사진 한 장을 놓고 “이것만으로 원인을 맞춰보라” — 학생들은 못 맞힌다(그게 포인트다).
- ② 결합 판정표 소개 (8분): 아래 ‘결합 판정 매트릭스’를 프로젝터로. 같은 노란 잎이라도 흙 상태(젖음 / 적정 / 마름)에 따라 과습 → 영양의심 → 판단보류로 결론이 바뀌는 걸 한 줄씩 짚는다.
- ③ 실제 앱 진단 (10분): 실시간 화면에서 스냅샷을 찍어 진단하면 사진(비전)과 그 순간의 센서값이 함께 넘어간다. 결과 화면의 ‘💧 수분 상태 판정’과 ‘🌱 AI 종합 소견’을 함께 읽는다.
- ④ 마무리 (5분): 규칙(💧) vs 생성(🌱) 비교 + AI 신뢰 규칙(가드레일) 이야기로 정리.

결과 화면 4층 읽는 법

- ① 종합 건강 평가 — 병해가 있나 없나를 한 문장으로(초록 / 노랑 / 빨강).
- ② 환경 항목 — 온도·습도·토양수분을 각각 적정 / 주의 / 위험으로.
- ③ 💧 수분 상태 판정 — 비전 근거(잎)와 센서 근거(흙)를 나란히 보여주고, 원인(과습 / 물부족 / …) बै지와 확신(높음 / 보통 / 낮음)을 붙인다. 이것이 ‘결합’의 결과다.
- ④ 🌱 AI 종합 소견 — 생성형 AI가 위 정보를 종합해 사람이 읽는 소견(한 문장)·근거(실제 수치 인용)·권장 조치를 쓴다. 확신이 낮으면 ‘확실하지 않아요’ बै지가 붙는다.

결합 판정 매트릭스

같은 잎 증상이라도 토양수분에 따라 결론이 달라진다 — 이것이 ‘두 데이터를 합치는’ 힘이다. (역병·시들음병은 병해 방제가 먼저라 수분 판정보다 우선한다.)

비전(잎 증상)	흙: 젖음(적정↑)	흙: 적정	흙: 마름(적정↓)	흙: 센서 없음
역병·시들음병(확신)	병해 (+과습 경고)	병해	병해	병해(비전만)
황화(노란 잎)	과습	영양·기타 의심	판단보류(신호 상충)	판단보류
잎끝마름	판단보류(상충)	영양·기타 (염류?)	물부족	판단보류
병징 없음(정상 확인)	과습 조기경보	정상	물부족 조기경보	정상(수분은 보류)
미탐지(신뢰도 낮음)	과습 조기경보	정상	물부족 조기경보	판단보류

‘조기경보’는 잎에 아직 증상이 없어도 흙 상태만으로 위험을 미리 알리는 것. ‘판단보류’는 근거가 엇갈리거나 부족해 억지로 결론 내지 않는 것.

관찰 포인트 (교사가 짚어줄 것)

- 같은 노란 잎, 다른 결론: 표에서 ‘황화’ 행을 가로로 읽어, 흙 상태만 바뀌어도 결론이 과습 → 영양의심 → 판단보류로 달라지는 걸 짚어줄 것 — 이 모듈의 핵심 장면.
- 💧와 🌱의 관계: 💧(수분 판정)는 정해진 표로 계산한 ‘규칙’의 답, 🌱(AI 소견)는 그 위에서 생성형 AI가 수치를 인용해 쓴 ‘문장’. 두 칸이 왜 따로 있는지 물어볼 것.
- AI가 ‘모른다’고 말하는 순간: 판단보류·‘확실하지 않아요’ बै지가 뜨는 케이스를 일부러 만들어(신호 상충) 보여줄 것 — “좋은 AI는 모를 때 모른다고 한다.”

예상 질문과 답

- Q. AI가 잎만 보고 과습인 줄 어떻게 알아요? → A. 몰라요. 흙(토양수분)을 같이 봐야 '노란 잎 + 젖은 흙 = 과습'을 알 수 있어요. 그래서 두 데이터를 합치는 거예요.
- Q. 왜 어떤 건 '판단보류'라고 나와요? → A. 신호가 엇갈리면(노란 잎인데 흙은 마름) 억지로 답하지 않아요. 의사도 애매하면 "검사 더 해보자"고 하죠.
- Q. 💧 수분 판정이랑 🌱 AI 소견은 뭐가 달라요? → A. 💧는 정해진 표(규칙)로 낸 답이라 같은 입력이면 늘 같은 답이 나와요. 🌱는 생성형 AI가 그 위에 수치를 인용해 사람 말로 써준 소견이에요.
- Q. AI가 없는 값을 지어내면요? → A. 그래서 "준 데이터에 없는 수치는 지어내지 마라, 센서가 응답 안 하면 그렇다고 말하라"는 규칙(가드레일)을 미리 정해뒀어요.
- Q. 센서값이 오래된 거면요? → A. AI가 "이건 마지막 측정값이고 지금은 센서가 응답하지 않는다"고 밝히도록 만들었어요(현재값처럼 인용하지 않게).

심화 활동 · 토론

- '결합의 힘' 표 채우기: [비전만 / 센서만 / 둘 다] 각각 무엇을 알 수 있는지 3칸 표로 정리 → 왜 합쳐야 하는지 스스로 발견한다.
- 판정 뒤집기: 같은 잎 증상에 흙 상태만 바뀌거나(젖음 / 적정 / 마름) 결론이 어떻게 달라지는지 매트릭스로 추적한다.
- 규칙 vs 생성 토론: 규칙(💧)은 항상 같은 답이라 설명하기 쉽지만 딱딱하고, 생성(🌱)은 유연하지만 검증이 필요하다 — 어느 쪽을 더 믿을까? 왜 둘 다 보여줄까?
- AI 윤리 한 걸음: "AI가 확신 없을 때 '모른다'고 말하게 만드는 것"이 왜 중요할까? (오진이 사람·농작물에 주는 피해와 연결)

진로 연계

- 이 모듈이 이어지는 직업: AI 신뢰성·안전 엔지니어, 프롬프트 엔지니어, MLOps(모델 운영), 데이터 과학자. "AI에게 규칙을 정해주고, 틀리지 않게 지키도록 만드는 사람들."

12. 마무리 (10분)

마무리 퀴즈 (인앱, 5분)

허브 상단의 "마무리 퀴즈" 버튼 → 7문항(OX·객관식)을 각자 풀고 자동 채점·해설 확인. 교사는 순회하며 점수를 확인한다. 문항 범위: 인증키 보안 / epoch / 수확체감·과적합 / 오탐 / ESP32 / 판단보류(신뢰도 75%) / 자동화 규칙 — 모두 본 수업에서 다룬 내용이다.

구두 예비 퀴즈 (인앱 퀴즈가 어려울 때)

1. 공공데이터를 쓰려면 먼저 무엇을 발급받아야 할까? → 인증키
2. AI 학습에서 '반복 횟수'를 뭐라고 부를까? → epoch
3. 학습이 잘 될수록 내려가는 값은? → loss (올라가는 값은 accuracy/정확도)
4. 센서 값이 저장되는 곳은? → 데이터베이스(Supabase)의 sensor_readings 테이블
5. 테이블에 새 데이터 한 줄을 추가하는 명령은? → INSERT
6. 앱은 무엇만 알면 카메라 영상을 볼 수 있을까? → 주소(camera_url)

소감 발표 (2~3명)

- "가장 신기했던 스테이션과 그 이유는?"

진로 연계 한마디

- 오늘 체험한 기술의 직업들: 스마트팜 엔지니어, AI 개발자, 데이터 라벨러, IoT 임베디드 개발자, 백엔드 개발자(데이터베이스). "오늘 버튼으로 해본 것을 코드로 직접 짜는 사람들"

13. 트러블슈팅 (교사용 빠른 참조)

증상	대처
화면이 멈추거나 이상함	새로고침(F5). 진행 데이터는 브라우저에 저장되어 있어 사라지지 않음
처음부터 다시 하고 싶음	허브의 "5 이 방 초기화 (처음부터 다시)" 버튼
방 번호를 잘못 선택함	허브에서 자기 방 번호를 다시 클릭 (이전에 하던 방 데이터도 그대로 남아 있음)
옆 친구와 같은 방 번호를 씀	PC가 다르면 데이터 충돌은 없음. 혼동 방지를 위해 배정표대로 재선택 안내
진행도(x/4)가 안 올라감	각 STEP의 완료 조건 확인 — 날씨: 데이터 받아오기까지, YOLO: 학습 완료까지, 센서: 측정값 1회 전송까지, 카메라: 라이브 켜기
인증키를 저장했는데 잠금이 안 풀림	붙여넣은 키가 너무 짧으면("인증키가 너무 짧아요...") 저장이 안 됨 — data.go.kr의 일반 인증키(Decoding)를 공백 없이 다시 붙여넣기
카드 ③에 '체험 모드' 오류가 뜸	학생 키 문제 아님. 서버에 기상청 키(KMA_SERVICE_KEY) 미설정 또는 기상청/네트워크 장애 신호 → 서버 키·네트워크 점검. 급하면 "체험 모드로 진행"으로 수업 계속
위치 권한 거부/실패	카드 ③의 "지역을 골라 호출하기"로 서울/대전/광주/부산/제주 선택 — 지역을 골라도 실제 기상청 호출
날씨가 계속 체험 모드로만 나옴	모든 학생이 서버 키 하나를 공유하므로 일 호출 한도 초과 가능 — 한도가 넉넉한 키 사용 권장, 초과 시 체험 모드로 우회
인터넷이 끊김	STEP 1은 인터넷 필수(복구 후 진행). STEP 2~4는 이미 열린 페이지에서 계속 동작
학생 진행 상황이 교사 화면에 안 보임	정상(원격 모니터링 기능 없음). 순회하며 각자 화면 상단의 진행도 바로 확인
수업 후 데이터 정리	각 PC에서 "모든 방 초기화 (선생님용)" 실행 (교사 PC 한 대에서 전체 정리 불가)

14. 부록 A — 용어 사전

용어	설명
공공데이터	정부·공공기관이 누구나 쓸 수 있게 무료로 공개한 데이터 (예: 기상청 날씨)
API	프로그램끼리 데이터를 주고받는 약속된 창구
인증키(serviceKey)	API를 쓸 자격이 있다는 것을 증명하는 열쇠 문자열. 공공데이터포털 (data.go.kr)에서 발급
초단기실황	기상청이 매시간 발표하는 현재 날씨 관측값(기온·습도·강수 등)
격자 좌표(nx, ny)	위경도를 기상청 예보 지도의 칸 번호로 바꾼 좌표
라벨링	AI 학습용 데이터에 사람이 정답(건강/병해 등)을 표시하는 작업
epoch	전체 학습 데이터를 처음부터 끝까지 한 바퀴 학습하는 횟수
loss	AI의 '실수 크기'. 학습이 잘 될수록 내려간다
accuracy(정확도)	AI가 정답을 맞힌 비율. 학습이 잘 될수록 올라간다
IoT	Internet of Things. 사물(센서, 가전 등)이 인터넷에 연결되는 것
ESP32	WiFi가 내장된 손가락만 한 소형 컴퓨터. 스마트팜 센서 보드의 두뇌
데이터베이스(DB)	데이터를 표(테이블) 형태로 저장·관리하는 시스템
Supabase	실제 개발자들이 많이 쓰는 클라우드 데이터베이스 서비스
INSERT	테이블에 새 데이터 한 줄(행)을 추가하는 데이터베이스 명령
스트리밍	영상·데이터를 내려받지 않고 실시간으로 이어 보는 방식
스냅샷	영상에서 캡처한 한 장의 정지 사진(프레임)
결합 진단(멀티모달)	사진·센서 수치처럼 서로 다른 종류의 데이터를 합쳐 하나로 판단하는 것
규칙 기반(rule-based)	미리 정한 조건표(이러이러하면 → 이 결론)로 판정. 같은 입력이면 항상 같은 답
생성형 AI(LLM)	사람처럼 문장을 만들어내는 AI(예: Claude). 근거를 사람 말로 서술한다
환각(hallucination)	AI가 없는 사실을 그럴듯하게 지어내는 오류
가드레일	AI가 지켜야 할 규칙(지어내지 마라 등)을 미리 정해 두는 안전장치
판단보류	근거가 엇갈리거나 부족할 때 억지로 결론 내지 않는 것
조기경보	앞에 아직 증상이 없어도 흠 상태로 위험을 미리 알리는 것
오탐(거짓양성, FP)	병이 아닌데 병이라고 잘못 알린 것. 반대로 병인데 놓친 것은 '누락(거짓음성)'
과적합(overfitting)	학습 데이터를 '암기'해버려 처음 보는 데이터에 오히려 약해지는 현상. epoch를

용어	설명
	무한정 늘리면 생길 수 있다
토양수분	흙 속의 물기 비율(%). 실제 농업에서 물 주기(관수)를 판단하는 기준 — 공기 습도와는 다르다
자동화 규칙	"만약 ~이면 → ~한다"로 조건과 행동을 묶은 것. 스마트팜 자동 제어의 기본 단위

15. 부록 B — 학생 활동지 (복사용, 1쪽)

이름: _____ 방 번호: _____번

활동 1. [STEP 1] 지역별 기온 비교

지역	기온(°C)	내 위치와의 차이
서울		
대전		
광주		
부산		
제주		

가장 따뜻한 지역과 이유 추측: _____

활동 2. [STEP 2] 미니 혼동행렬

회차	실제(내가 본 앞)	AI 판단	신뢰도(%)	맞음/틀림
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

오탐(건강한 앞→병해) 횟수: _____회 · 정밀도 = 병해 판정 중 실제 병해 비율 = _____%

활동 3. [STEP 3] 이상치 탐정

짜이 몰래 넣은 입김 이벤트 시각: _____ : _____ / 그래프에서 찾은 단서: _____

활동 4. [캡스톤] 나의 자동화 규칙

규칙(만약 ~이면 → ~한다)	사용한 데이터(날씨/AI/센서/카메라)	왜 그 조건에 그 행동인가(근거)